Quant Agents with LLMs

A Multi-Agent Trading System with Quant, Hybrid, and LLM Operating Modes

AIMS 5790 – Project Presentation

The Chinese University of Hong Kong

April 22, 2026

Outline

Part 1. Motivation and Research Problem

Why Multi-Agent Trading with LLMs?

Building a modern trading system is an **engineering** problem before it is a forecasting problem.

A credible research platform must simultaneously:

- combine **heterogeneous alpha sources** (price, volatility, news);
- preserve **temporal causality** with strict as-of data access;
- allocate capital under **explicit risk constraints**;
- support controlled comparison between **quant, LLM, and hybrid** decision styles.

Research question: can the same infrastructure host rule-based, LLM-based, and mixed agents in a reproducible and leak-safe way?

Four Engineering Problems

1. **Alpha fusion.** Normalize and combine momentum, mean-reversion, regime, and news signals with different reliabilities.
2. **Temporal realism.** Ensure only data visible at time t influences decisions at time t.
3. **Safe capital allocation.** Turn symbol-level conviction into portfolios with exposure, concentration, and drawdown controls.
4. **Controlled mode comparison.** Keep the surrounding architecture fixed while swapping quant / LLM components in and out.

These translate into concrete research questions:

- How should multiple alpha sources be normalized and fused?
- Does regime-aware weighting beat static weighting?
- How to add textual news without destroying causality?
- Which pipeline stages are safe to hand over to LLMs?
- How to structure parameter search to avoid overfit artifacts?

Part 2. Related Work

TradingAgents vs AI-Trader vs Ours

	TradingAgents	**AI-Trader**	**Our project**
Top-level structure	Hierarchical role pipeline modeled after a trading firm	Autonomous multi-model arena with tool-driven agents	Typed decision pipeline (orchestrator + portfolio + execution)
Early stage	Analyst team (fundamentals, sentiment, news, technical)	Standalone models running concurrently	Market, mean-reversion, news, regime, risk agents
Middle stage	Bullish / bearish debate, then trader synthesis	Tool-driven autonomous research	Grouped signal fusion inside a portfolio step
Risk	Committee after researcher synthesis	Property of each model's policy	**Two-layer** soft scale + hard veto
Execution	Simulated exchange	Competitive performance tracking	Backtest engine + persistent local artifacts
Paradigm	Role-based reasoning hierarchy	Autonomous multi-model trading arena	**Typed, reproducible decision-state transitions**

Our system is closest to a **decision engine**: information moves through structured data objects and deterministic portfolio logic.

Design Takeaways From Prior Work

From TradingAgents we take:

- separation of analysis, trading, risk, and portfolio into distinct responsibilities;
- explicit risk and portfolio stages downstream of signal generation.

From AI-Trader we take:

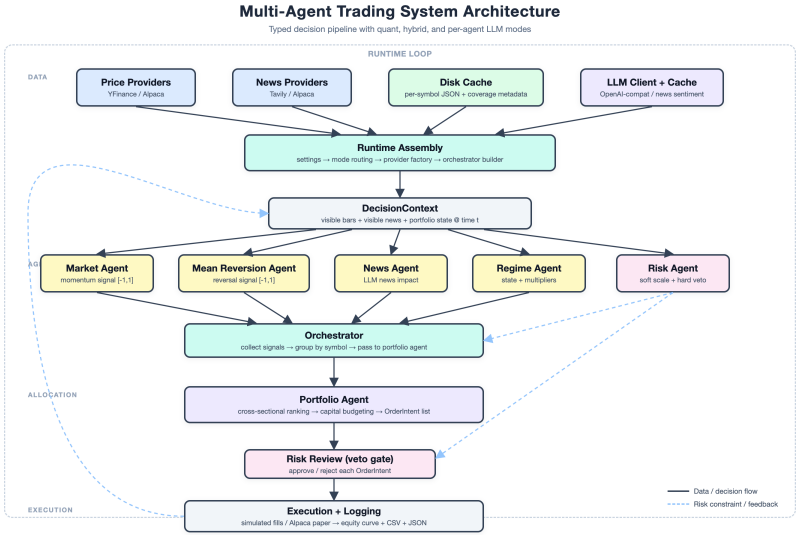
- comparable agent runs on the **same market window**;
- a benchmark-oriented evaluation loop.

What we do differently:

- reproducible backtesting with **typed intermediate objects** instead of long-form reasoning traces;
- a single typed execution path, with configuration switching between quant, hybrid, partial-LLM, and full-LLM modes.

Part 3. System Design and Implementation

High-Level Architecture



Providers, runtime assembly, agents, orchestrator, and execution adapters are separated by explicit interfaces.

Domain Model and Orchestration

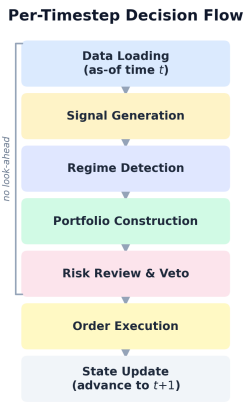
Typed decision objects.

- `Bar`, `NewsEvent` – inputs;
- `Signal` – agent outputs with confidence and metadata;
- `OrderIntent`, `Fill` – proposed and realized trades;
- `Portfolio`, `DecisionContext` – state wrappers.

Orchestrator flow.

1. Build `DecisionContext` from visible bars / news / portfolio state.
2. Collect `Signals` from alpha, regime, risk agents.
3. Portfolio agent converts grouped signals to `OrderIntents`.
4. Risk agents may veto orders via `review_orders()`.
5. Execution adapter produces `Fills`.

Decision Flow per Time Step



At each timestep t the pipeline walks a fixed, **leak-safe** sequence of stages:

1. **Data loading** — only bars / news visible at t.
2. **Signal generation** — momentum, mean-reversion, news.
3. **Regime detection** — state + allocation multipliers.
4. **Portfolio construction** — rank, budget, target positions.
5. **Risk review & veto** — soft scale + hard rejection.
6. **Order execution** — simulator or Alpaca paper.
7. **State update** — advance clock to $t + 1$.

The **"no look-ahead"** bracket covers the decision stages: any future information is strictly unavailable inside that block.

Effective Operating Modes

Profile	Behavior
Pure quant	Rule-based market, mean-reversion, regime, risk, portfolio; `NullNewsProvider` disables news entirely.
Hybrid (quant + LLM news)	Quant agents stay active; `NewsAgent` added as *confirmation-only* layer inside the portfolio step.
Partially LLM	Per-agent overrides: e.g. market / regime / risk / portfolio selectively LLM; the rest remain quantitative.
Fully LLM	All major decision agents replaced with LLM versions; LLM news also enabled.
Fallback	Any LLM-configured agent without an API key logs a warning and reverts to the quant implementation.

This **mode matrix** is what makes the project a comparison framework, not a single trading strategy.

Agent Implementations

Component	Quant	LLM	Main output
Market	`MarketAgent`	`LLMMarketAgent`	Momentum score $\in [-1, 1]$
Mean reversion	`MeanReversionAgent`	`LLMMeanReversionAgent`	Mean-reversion score $\in [-1, 1]$
News	—	`NewsAgent`	News impact score + metadata
Regime	`RegimeAgent`	`LLMRegimeAgent`	State label + policy multipliers
Risk	`RiskAgent`	`LLMRiskAgent`	Risk scale + optional order veto
Portfolio	`SimplePortfolioAgent`	`LLMPortfolioAgent`	Target positions $\rightarrow$ order intents

Interface parity. LLM agents satisfy the same output contracts as their quant counterparts, so downstream allocation, risk review, and execution logic are unchanged.

Quantitative Agent Formulas (Overview)

Momentum (volatility-normalized trend):

$$s_{i,t}^{\text{mom}} = \tanh\left(\frac{(M_s(t) - M_l(t))/M_l(t)}{\max(\sigma_v(t),\, A_a(t),\, 10^{-4})} \Big/ \lambda_{\text{mom}} \right).$$

Mean reversion (volatility-normalized reversal):

$$s_{i,t}^{\text{mr}} = \tanh\left(\frac{-(P_t - M_L(t))/M_L(t)}{\max(\sigma_v(t),\, A_a(t),\, 10^{-4})} \Big/ \lambda_{\text{mr}} \right).$$

Regime state rule:

$$\text{state}_{i,t} = \begin{cases} \texttt{crisis}, & \sigma_t \geq \theta_\sigma \cdot c_\sigma, \\ \texttt{low-liquidity}, & \ell_t < \theta_\ell, \\ \texttt{trend}, & |g_t| \geq \theta_g, \\ \texttt{chop}, & \text{otherwise}. \end{cases}$$

Regime emits **policy multipliers** (weights, gross exposure, threshold, turnover).

Portfolio Construction: Alpha Fusion

For symbol i at time t, with regime-adjusted source weights $w'_{i,k}$ and per-source score $\hat{s}_{i,k}$:

$$\alpha_{i,t} = \frac{\sum_{k \in \mathcal{A}_i} w'_{i,k}\, \hat{s}_{i,k}}{\sum_{k \in \mathcal{A}_i} w'_{i,k}}.$$

The soft risk scale $r_{i,t} \in [0, 1]$ multiplies conviction:

$$c_{i,t} = r_{i,t}\, \alpha_{i,t}.$$

Candidates are **cross-sectionally ranked** by

$$\text{strength}_{i,t} = |c_{i,t}| \cdot \text{confidence}_{i,t} \cdot G_{i,t},$$

then capital is allocated with inverse-volatility budgeting under gross-exposure, per-position, and turnover caps.

Two-Layer Risk Control

Risk is **not** a single late-stage committee but two explicit computations.

Soft layer (during signal fusion).

- Drawdown scale: $\max(0,\, 1 - d_t/D)$ with $d_t = (E^{\max} - E_t)/E^{\max}$.
- Exposure scale: $\max(0,\, 1 - e_{i,t}/p)$.
- Product becomes the per-symbol risk multiplier $r_{i,t}$.

Hard layer (after portfolio construction).

- Block new buy orders if $d_t \geq D$;
- Reject orders that push exposure above $1.5\,p$;
- LLM risk agent acts only at this hard-veto stage.

Effect: risk is enforced both *before* and *after* allocation.

Data, Caching, and Runtime Applications

Providers & caching.

- Prices: Yahoo Finance / Alpaca. News: Tavily / Alpaca.
- Disk caches per symbol with **range-coverage metadata** $\Rightarrow$ skip upstream API when in range.
- LLM news answers are additionally hashed & cached by model, namespace, and payload.

Command-line entry points.

- `qa-backtest` – historical backtest over a date range.
- `qa-paper` – persistent Alpaca paper-trading loop.
- `qa-walkforward` – month-by-month segmented evaluation.
- `qa-optimize` – Optuna search + optional anchored walk-forward validation.

Part 4. Research Methodology

Backtest Engine: Temporal Controls

As-of data access.

- Preload bars and news over the full range, then advance an explicit simulation clock.
- Only bars / news with timestamp $\leq t$ are exposed to agents.
- If the latest bar is timestamped exactly at t, it is **removed** from the decision context.
- Orders then execute at that current bar's **opening** price.

No-look-ahead test. Verified that a news event published at a future time remains invisible until the simulation clock reaches that timestamp.

$\Rightarrow$ reduces inflated performance caused by temporal leakage.

Optimization and Walk-Forward Evaluation

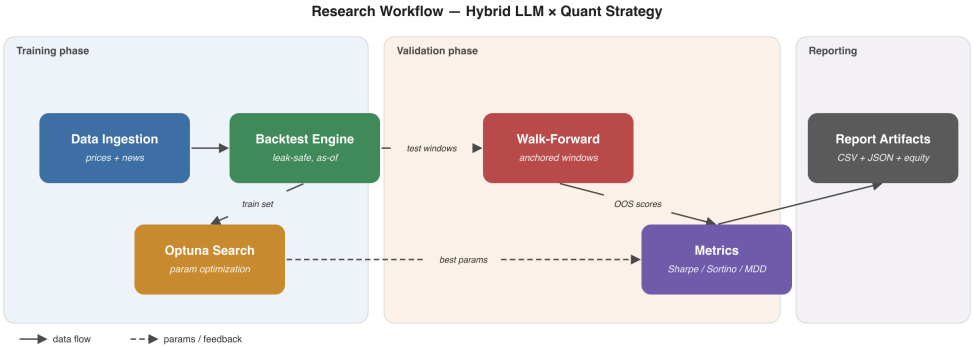
Optuna-based parameter search.

- Warmup extension derived from lookbacks before the official start date.
- In-memory price provider to avoid repeated I/O.
- **News parameters are held fixed** – search is intentionally a pure-quant loop, with `NullNewsProvider`.

Anchored walk-forward validation.

- Last 30% of the date range split into non-overlapping test windows.
- Training windows anchored at the global start and grow as tests advance.
- Top in-sample candidates re-evaluated on held-out windows.
- Final config chosen by **average out-of-sample metric**.

Research Workflow



Price data feeds the backtest engine with explicit temporal controls, which supports parameter search, walk-forward testing, and reporting.

Part 5. Experimental Results

Implementation Status

Area	Verified behavior
Runtime configuration	Env-driven settings; global + per-agent mode overrides
Domain model	Shared bar / news / signal / order / fill / portfolio objects
Data providers	Yahoo Finance, Alpaca prices; Tavily, Alpaca news
Caching	Per-symbol disk caches + range-coverage metadata
Quant agents	Market, mean-reversion, regime, risk, portfolio logic
LLM agents	Market, mean-rev, regime, risk, portfolio, news analysis
Backtest engine	Timeline simulation with as-of reads, current-bar exclusion
Execution	Simulated fills; Alpaca paper execution
Optimization	Optuna search + optional anchored walk-forward
Reporting	CSV fill / equity logs + JSON summaries

Regression suite: **44 passed / 0 failed** in the final pass.

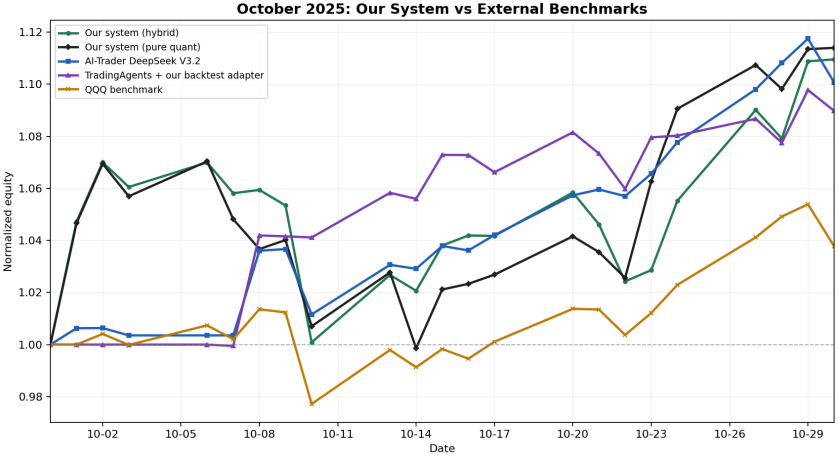
Headline Financial Results

Experiment	Final equity	Ret (%)	Sharpe	Sortino	Max DD
October 2025 (initial capital 100,000)					
Pure quant	**111,399**	**11.40**	**4.12**	**7.33**	**0.067**
Hybrid quant + LLM-news	110,956	10.96	3.79	6.29	0.065
Best full-LLM run	101,085	1.09	2.23	5.15	0.005
AI-Trader `DeepSeek V3.2`	11,007	10.07	6.17	11.47	0.024
TradingAgents + our adapter	10,898	8.98	5.30	15.49	0.020
QQQ benchmark	103,780	3.78	2.42	3.31	0.036
March 2020 (crisis month, initial capital 100,000)					
Hybrid quant + LLM-news	107,558	7.56	1.89	2.59	0.146
Pure quant, news disabled	**109,688**	**9.69**	**2.37**	**3.29**	**0.119**
QQQ benchmark	88,924	-11.08	-1.21	-1.61	0.224

Note: AI-Trader and TradingAgents rows use 10,000 initial capital; percentage returns are directly comparable.

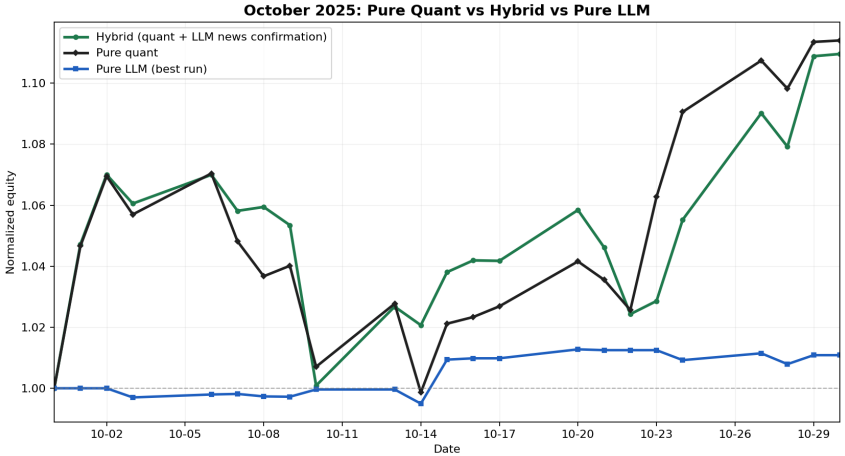
Quant-driven configurations **beat QQQ**; the full-LLM run trails QQQ in October 2025.

October 2025: External Benchmark Comparison



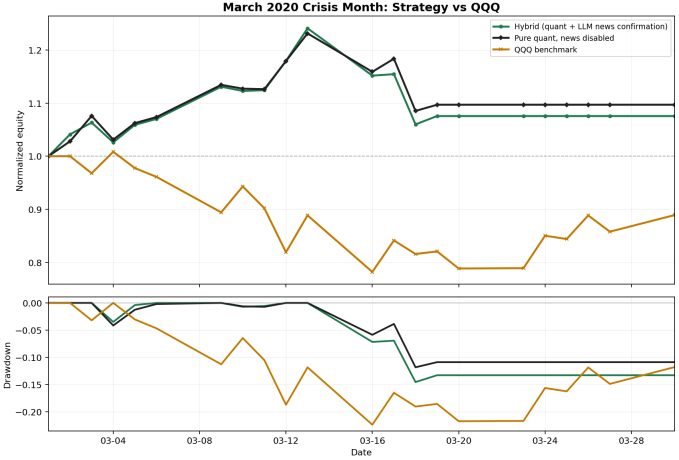
Hybrid 10.96% > AI-Trader `DeepSeek V3.2` 10.07% > TradingAgents + adapter 8.98% > QQQ 3.78%. Pure-quant (11.40%) is slightly ahead of hybrid on this month.

October 2025: Mode Comparison



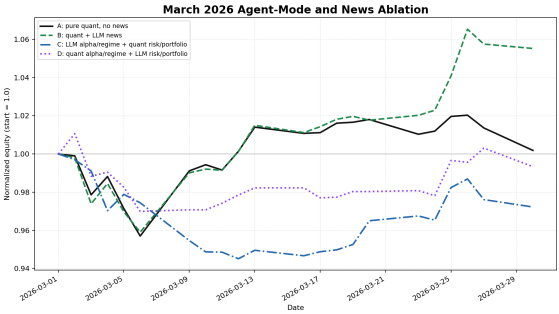
Pure quant **11.40%** vs hybrid **10.96%** vs pure-LLM **1.09%**. The pure-LLM path is essentially flat; quant-driven configurations compound much more aggressively.

March 2020: Crisis-Month Stress Test



Under a crisis regime, both strategies stay profitable while QQQ falls 11.08%. However pure quant (9.69%, DD 11.85%) outperforms hybrid (7.56%, DD 14.57%) – the news overlay is *regime-dependent*.

March 2026: Agent-Mode Ablation



Same symbols, dates, capital, parameters – only **agent routing** changes.

Run	Config	Ret (%)	Sharpe
A	Pure quant	0.18	0.21
B	**Quant + LLM news**	**5.52**	**3.38**
C	LLM alpha/regime	-2.78	-2.29
D	LLM risk/portfolio	-0.67	-0.55

Findings:

- LLM news **helps** a quant core (A→B: +5.3%).
- Replacing *alpha/regime* with LLM hurts most (Run C).
- LLMs work best as bounded overlays, not core replacements.

Why the News Layer Can Underperform

Adding the LLM-news confirmation layer does *not* automatically improve returns:

- **Lag.** By the time a headline is published, filtered, summarized and scored, the market may already have repriced the information.
- **Noise.** Headline-level news is incomplete, especially when macro conditions dominate individual company narratives.
- **Confirmation veto.** Our implementation is confirmation-only, so it can delay or shrink strong quantitative trades whenever headline sentiment disagrees with a faster price-based signal.
- **Microstructure blindness.** LLM analysis is textual; it can read direction correctly while still mistiming the tradable impact.

$\Rightarrow$ news is an **interpretive overlay**, not a guaranteed return booster.

Key Findings

1. On Oct 2025, quant-driven configurations **outperform** AI-Trader `DeepSeek V3.2`, TradingAgents + adapter, and QQQ in cumulative return.
2. Pure-quant $\approx$ hybrid $\gg$ pure-LLM: LLMs are most effective as **confirmation / filtering** layers rather than full replacements.
3. The strategy remains profitable through the March 2020 crisis while QQQ drops 11%; yet the news overlay did *not* help in that regime.
4. March 2026 ablation shows LLM news **can help** a quant core in some regimes; replacing core alpha / regime logic with LLM hurts.
5. No-look-ahead, caching, and walk-forward validation make the results **reproducible** rather than merely impressive.

Part 6. Discussion and Future Work

Strengths and Limitations

Strengths.

- Engineering decomposition: typed objects, pluggable providers, explicit risk layers, multi-backend execution.
- Dual-mode design: pure quant / hybrid / partial / full LLM within *one* framework.
- Causality and overfitting controls: as-of reads, current-bar exclusion, walk-forward evaluation.

Limitations.

- Optimization currently excludes the news / qualitative parameters.
- The LLM portfolio path still has model-dependent sparse selection.
- Headline results cover a small set of months, not a full multi-regime panel.
- `qa-walkforward` is a reporting loop, not a retraining walk-forward optimizer.

Future Work

- Publish **mode-by-mode ablations** side by side across many months and market regimes.
- Add **programmatic post-LLM clipping** for portfolio constraints.
- Extend optimization to a **controlled subset** of qualitative parameters.
- Promote the execution router to a real **live-trading backend** (beyond the current Alpaca paper layer).
- Analyze whether LLMs improve **out-of-sample robustness** or only narrative richness.

Part 7. Conclusion

Conclusion

`Quant Agents with LLMs` is a causality-aware, configurable trading research platform.

What we built. Backtest engine with explicit temporal controls, disk-cached providers, dual quant + LLM agent stack, portfolio construction layer, two-layer risk, Optuna optimization, walk-forward validation, structured reporting, and Alpaca paper-trading execution.

What we found. Hybrid designs in which *quant logic controls allocation* and *LLMs act as bounded overlays* remain competitive with strong external baselines and resilient under crisis – while replacing the core with LLMs underperforms.

What's next. Broader empirical coverage, unified optimization, and robust live execution.

Part 8. References

References I

Tauric Research.
TradingAgents.
GitHub: github.com/TauricResearch/TradingAgents, 2025.

HKUDS.
AI-Trader.
GitHub: github.com/HKUDS/AI-Trader, 2025.

Wayland Zhang.
Quantitative Trading Book.
waylandz.com/quant-book, 2025.

T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama.
Optuna: A next-generation hyperparameter optimization framework.
In *KDD*, 2019.

Thank you for your time!

Questions & discussion welcome.